《软件设计与规范》课程教学大纲（2026 秋季修订版）

Software Design and Conventions

一、课程基本信息

1.课程名称：软件设计与规范（Software Design and Conventions）

2.课程编码：50500098230

3.学时与学分：56 学时，本课程由两位教师分段授课，本大纲覆盖其中 28 学时的教学部分；学分：3.5；

4.适用专业：软件工程、物联网、人工智能、大数据

5.考核办法：考查、五分制、成绩评定依据见附表

6.先修课程：软件工程概论，Java 程序设计，数据结构，Web 前端开发，Web 后端开发；后继课程：软件验证与确认，软件设计综合课程设计

7.课程责任单位：计算机学院

二、课程简介

《软件设计与规范》是软件工程的专业核心课，授课时间正处于基础课到专业课的过渡阶段，学生从编程语言、数据结构与算法的学习迈入工程化实践，本课程承上启下，为后续解决复杂问题的综合性实践奠定基础。本课程的任务是使学生了解软件工程国内外标准与规范，软件生命周期与软件开发的一般过程，经典开发方法，掌握主流技术和工具；并掌握 AI 时代的开发新范式 AI for SE：把生成式 AI 作为受规范约束的开发协作者，学会以提示词工程规范与 AI 组成结对、以工程标准验证 AI 产出，理解含 AI 组件软件的体系结构与设计原则，培养学生解决工程问题的思维能力和实践能力，熟悉团队分工与角色职责，训练团队协作的综合素质。

本课程与后继课《软件验证与确认》形成分工：本课主责“用 AI 做开发”（AI for SE）及含 AI 组件的设计，后继课主责“对含 AI 的软件做质量保障”（SE for AI）。本课程的核心内容包括：

（1）标准与规范：IEEE 软件工程知识体系 SWEBOK 的软件开发部分，中国软件工程知识体系 C-SWEBOK 的软件开发部分，GB/T 11457 软件工程术语，GB/T 8566 软件生存周期过程，ISO/IEC 25010 软件质量模型，GB/T 25000.10 系统与软件质量模型，GB/T 8567 计算机软件文档编制规范，大小写约定，类型、类型成员、参数、资源的命名规范等。面向 AI 的规范：提示词模式、AI 生成代码使用与标注规范。

（2）基本理论：软件生存周期、软件开发过程模型、软件开发与软件工程其他活动的关系、软件设计过程、软件设计质量、软件体系结构设计、类设计、设计精化、设计模式、面向对象软件设计方法及基本原则、基于 UML 的分析与设计过程、用例 UseCase 分析、数据模型设计、用户界面设计、部署模型设计，Clean Code 概念与原则。含 AI 组件的体系结构与面向不确定性组件的设计原则，AI 使用与学术诚信规范。

（3）开发技术：依赖注入 DI、对象关系映射 ORM、面向切面编程 AOP、单元测试、冗余代码检测与重构。AI 结对编程与 AI 辅助重构。

三、课程目标

本课程目标为培养学生解决复杂问题能力。从 CDIO 工程教育理念出发，遵循需求、设计、编码到测试的软件工程实践逻辑，采用基于软件开发过程、嵌入高阶思维训练的增量式教学模型，将 CBL、TBL 与 PBL 相结合，达成教学目标。通过案例教学 CBL，掌握科学计算、信息管理系统两类软件的开发方法和技术，掌握开发工具的使用原理和方法，理解其局限性；运用团队教学 TBL，体验软件工程团队核心角色，引导学生职业定位，依据学习效能的自我感知，发掘职业兴趣，增强学习动力；采用问题教学 PBL，实施高阶思维训练，对复杂软件工程问题的解决方案进行比较和评价。掌握开发工具（含 AI 编码助手）的使用原理和方法并理解其局限性，能对复杂软件工程问题的解决方案（含 AI 给出的方案）进行比较和评价。

课程目标 1：熟悉软件工程国家标准与国际标准体系，理解流程规范与内容规范的区别；能够

比较、评价设计方案（含 AI 给出的方案），指出其优势与不足。

课程目标 2：掌握模块化设计原则、体系结构设计方法与架构模式，以及设计模式、重构与系统演化方法；能在既有系统的约束下诊断设计缺陷，提出改进方案，并借助 AI 实施、核实其产出，了解影响设计目标和技术方案的各种因素。

课程目标 3：掌握并遵守编码规范、接口契约规范与协作记录规范，掌握测试的基本要求与代码编译部署的方法，理解规范条款背后的设计理由；掌握集成开发环境、构建与测试工具、依赖注入容器、对象关系映射框架和架构约束检查工具的使用原理与方法，能用它们验证设计与代码，并理解它们的局限。

课程目标 4：掌握 AI for SE 的一般过程、AI 时代的设计原则、面向不确定性组件的应对措施，以及 AI 输出的核对内容；能对 AI 给出的结论与代码作出采纳、部分采纳或驳回的判断，并理解 AI 编码助手的局限。

四、课程目标与毕业要求对应关系

表 1 **课程目标与毕业要求映射表**

毕业要求	毕业要求指标点	课程目标
1.工程知识	1.4 能够运用软件工程专业知识和数学模型方法对复杂软件工程问题的解决方案进行比较和评价。	1
3.设计/开发解决方案	3.1 掌握软件工程设计和软件产品开发全周期的基本设计/开发方法和技术，了解影响设计目标和技术方案的各种因素。	2
5.使用现代工具	5.1 掌握现代软件工程中常用的现代仪器、信息技术工具、工程工具和模拟软件的使用原理和方法，并理解其局限性。	3、4

五、课程内容

第一章：标准与规范（2 学时，支撑课程目标 1）

1.教学重点和难点

重点：软件工程标准规范体系与主要标准；流程规范与内容规范；提示词模式作为人与 AI 之间的规格说明；质量模型的可维护性子特性及其判定。

难点：标准允许按项目剪裁，以及标准体系内部的版本差；把与 AI 协作纳入工程规范范畴的意识转变。

2.教学基本要求

理解：课程教学目标、考核方法与教学设计；软件工程的基本内涵；流程规范规定按什么步骤做，内容规范规定产物必须写成什么样。

了解：软件工程标准体系与主要国家标准；线上学习资源与学习方法。

掌握：质量模型的可维护性子特性；本课程对 AI 协作与产出验收的规定。

3.教学基本内容

课程导入：教学目标、教学设计（方法与过程模型）、教学方式、考核方法与 AI 使用规范。

软件工程标准规范体系：标准的层次，软件工程主要国家标准及其版本演进。

规范：流程规范与内容规范，标准可以按项目剪裁。

质量模型：产品质量模型与使用质量模型，使用可维护性的子特性（模块化、可复用性、易分析性、易修改性、易测试性）来比较、评价设计方案。

需求如何转为设计方案：质量特性是非功能需求的语言，设计是对它的回应；把性能、安全、易用性、可靠性等质量需求转成设计手段与代价，技术说明写成回应的需求、优点、限制与不足。

AI for SE：提示词模式，如 Persona、Few-shot、Template、Question Refinement 等；AI 协作工作流：任务分解 → 上下文与边界界定 → 先写验收准则 → 人工审查与修订记录；AI 输出的核对内容：核对引用是否存在、核对运行时事实、区分原理与实现、拿原则与规范作依据；先写验收准则、标注 AI 的使用范围、对产出承担解释责任。

4.教学建议

结合课程选定的棕地项目，安排一次当场判断；

线上预习标准文本与规范条目，课堂只讲规范之间的关系与取舍；

第一次课公布本课程对使用生成式 AI 的规定，并说明协作记录与提交说明的写法。

5.本章参考资料

[15] SWEBOK Guide V4.0 第 4 章；[16] GB/T 25000.10-2016 第 4 章（承接 ISO/IEC 25010:2011，八特性）；[17] GB/T 8566-2022；[18] GB/T 8567-2006；[19] ISO/IEC 25010:2023（九特性，新增系统安全 Safety，Usability→Interaction Capability、Portability→Flexibility）；[20] ISO/IEC/IEEE 12207:2026；[29] White 等 2023（提示词模式）；[42] Vanderbilt《Prompt Engineering for ChatGPT》；[44] Northeastern CS4530《Policy on Use of AI》；[47] 课程组《AI 协作与产出验收规范》。

第二章：运行环境（4 学时，支撑课程目标 1、3）

1.教学重点和难点

重点：软件运行环境的主要形态，单体、分布式、容器化，云服务的层次；各运行形态的适用范围与工程用法。

难点：运行环境作为体系结构的约束来源：环境变了，部署方式与物理视图随之变化；在受约束的环境里搭建并运行一个既有系统。

2.教学基本要求

了解：开发环境与运行环境；虚拟化与云计算的层次；基础设施即服务 IaaS、平台即服务 PaaS、软件即服务 SaaS；单体、分布式与容器化部署。

理解：运行环境与体系结构的关系，运行环境的演化脉络。

掌握：能在给定的环境中搭建、编译并运行棕地项目；能借助 AI 排查环境类故障，并按 AI

输出的核对内容核对 AI 给出的排查建议。

3.教学基本内容

软件运行环境：开发环境与运行环境；虚拟化与云计算层次（基础设施即服务 IaaS、平台即服务 PaaS、软件即服务 SaaS）；单体、分布式、容器化；运行环境的演化脉络。各运行形态按它回应的非功能需求与带来的代价讲授，并与软件发展过程或质量特性相联系。

实验环境：数据库、缓存、消息中间件、检索组件等在系统中的位置与边界；容器化编排方式；配置与密钥的管理要求。

棕地项目选取原则与使用约定：业务背景，模块划分、构建方式、启动方式；选取标准与全课程统一使用的约定，以及不得修改的部分。

软件设计的基本概念：抽象与逐步求精；模块化与信息隐藏，按"藏起什么设计决定"划分模块；内聚与耦合，追求高内聚、低耦合。结合本项目的模块依赖图讲授。

需求如何转为设计方案：以本项目与微服务版的对照为例，说明各组件回应哪条非功能需求、付出什么代价；技术说明写成回应的需求、优点、限制与不足。

AI for SE：AI 辅助环境搭建与故障排查。

4.教学建议

以棕地项目的运行环境为例讲物理视图与部署形态的差异；

本讲安排一次环境搭建实验，学生在本地或给定环境中把系统跑起来，教师现场演示整站运行，避免后续实验把时间耗在环境上。

5.本章参考资料

[21] NIST SP 800-145；[22] Parnas 1972；[27] Le 等 2015；[43] UC Berkeley《Large Language Model Agents》；[45] 课程实例项目 mall；[46] mall 的微服务版本 mall-swarm；[47] 课程组《AI 协作与产出验收规范》；[48] 课程组《mall / mall-swarm 产品拆解》。

第三章：体系结构设计（8 学时，支撑课程目标 1、2、3）

1.教学重点和难点

重点：需求分析与需求验收准则；视图建模；分层与架构边界；设计原则与 SOLID；表现层架构模式；依赖注入与控制反转；对象关系映射。

难点：把依赖方向写成可重复执行的检查规则；接手既有系统时先判断不可修改的部分；区分设计原理与具体框架的实现。

2.教学基本要求

理解：视图与利益相关者关注点的对应关系；Tier（部署层次）与 Layer（逻辑层次）的区别；架构决策记录的结构；对象与关系数据库之间的阻抗失配；几种数据源架构模式的差别；工作单元、标识映射、延迟加载各自解决什么问题。

掌握：需求验收准则的写法；分层的职责与依赖方向；识别分层缺陷；架构约束的表达与检查；不得修改部分的识别；表现与业务分离的原则；控制反转与依赖注入的方式及其与服务定位器的比较；能评价既有系统中表现层组织、依赖装配和数据访问方式的设计取舍。

3.教学基本内容

需求分析：参与者与用例、业务规则、需求验收准则；使用追问写清楚需求。

视图建模：逻辑视图、进程视图、开发视图、物理视图与场景；用 UML（统一建模语言）类图、顺序图、包图、部署图分别表达逻辑视图、进程视图、开发视图与物理视图；体系结构风格与架构边界的划分方式，各自的依赖方向。

分层与架构约束：违背分层的常见形态；把依赖方向写成可自动执行的检查规则；应防止后续修改违背约束。含 AI 组件的体系结构，如模型即服务（MaaS）接入、检索增强生成（RAG）、智能体编排；AI 组件应按不可靠外部依赖对待，借鉴分层解耦思想，架构上的应对措施：隔离、超时、重试、缓存与降级。

表现层架构模式：表现逻辑与领域逻辑分离，状态通过事件同步；表现层模式 MVC（模型—视图—控制器）、MVVM（模型—视图—视图模型）的组成、依赖方向与可测试性。

不得修改的部分与架构决策记录：接手既有系统时先判定不可修改的部分；视图说明系统长什么样，架构决策记录说明当初这样定的理由。

设计原则与 SOLID：常见设计原则（KISS、DRY、Tell Don't Ask、YAGNI、SoC，以及不过度设计）各一句定义与一处本项目证据；SOLID（面向对象设计的五条基本原则：单一职责、开闭、里氏替换、接口隔离、依赖倒置）各一句定义与一处本项目证据，依赖倒置与依赖注入相接。原则之间会冲突，取舍要写出理由。

依赖注入与控制反转：依赖注入是依赖倒置的实现机制；配置与使用分离；注入方式与组合根；依赖注入对比服务定位器。

对象关系映射：数据绑定的原理；阻抗失配；数据源架构模式的适用场合；工作单元、标识映射、延迟加载；全映射方案与 SQL 映射方案的取舍。

AI 产出的验收方法：AI 产出的验收准则（准确率/拒答率/延迟/成本/幻觉容忍度 → 阈值）；评测集构建方法；对抗用例设计（越界输入、提示注入尝试）；提示注入与最小权限原则；风险台账与验收报告结构；放行 / 降级 / 驳回的判定依据。

4.教学建议

使用棕地项目的模块依赖图、生成代码边界和共用表结构讲不得修改的部分；用同一业务存在两处实现这类现象，讨论它属于架构缺陷还是合理的边界；

通过实例讲解依赖注入原理，使用 Spring、CommunityToolkit 等不同实现技术进行横向比较。

通过实例讲解数据源架构模式的选择与代价；线上预习框架的接口与配置。

5.本章参考资料

[1] 《软件设计与体系结构》；[7] Fowler《企业应用架构模式》第 1、3、4 章；[8] Ford 等《演进式架构》；[9] Evans《领域驱动设计》第 4 章；[10] Vernon《实现领域驱动设计》第 4 章；[11] Newman《微服务设计》；[25] Kruchten 1995；[36] Fowler《GUI Architectures》；[37] Fowler《IoC 容器与依赖注入》；[38] Microsoft《Model-View-ViewModel》；[49] Nygard《Documenting Architecture Decisions》；[39] Spring 参考文档“Dependency Injection”。

第四章：软件设计、重构与系统演化（8 学时，支撑课程目标 2、3）

1.教学重点和难点

重点：模式的效果；从坏味道到重构再到模式；特征测试与接缝；逆向工程与再工程。

难点：模式是重构的结果，不是设计的起点；没有测试保护就不能修改既有代码；在维护、重构、再工程和替换之间作选择。

2.教学基本要求

理解：模式；常见坏味道；软件演化规律；从业务价值和系统质量角度评价遗留系统；再工程的过程。

掌握：用策略、状态、模板方法等模式消除对应的坏味道，并说明引入模式的代价；为既有代码编写特征测试，利用接缝在测试保护下安全修改；能读懂既有系统的结构并作出改造方案。

3.教学基本内容

软件设计过程：软件设计的一般过程（两层迭代），软件设计的主要活动（设计计划、体系结构设计、界面设计、模块/子系统设计、数据模型设计、过程/算法设计），各活动配一个典型例子；以订单为例，讲解抽象与逐步求精过程，说明软件设计过程与主要活动；软件设计的质量，软件设计规格说明，软件设计评审。抽象与逐步求精、模块化与信息隐藏、内聚与耦合已在第二章讲授；SOLID 已在第三章讲授，此处只做应用与判定。

设计模式：模式的四个要素（名称、问题、解决方案、效果）；评价模式，得到什么、失去什

么；策略模式、装饰模式、状态模式与模板方法各用四个要素写一遍，并说明各自消除的坏味道（条件分支密集时用策略或状态模式，重复的处理流程用模板方法）。

安全修改既有代码：先用特征测试固定现有行为，再通过接缝进行修改；框架的代理机制与同类方法自调用会让注解失效。

逆向工程与再工程：从代码恢复设计与体系结构；遗留系统的评价；演化策略的选择：维护、重构、再工程、替换；体系结构的恢复与改善，代码重构与数据重构；再工程的过程：逆向工程、设计恢复、重构、正向工程。

AI for SE：用 AI 辅助识别坏味道、生成重构方案与特征测试；评估 AI 给出的重构方案，先判断它是否触及不得修改的部分、是否夹带无关修改，再验证设计。

4.教学建议

坏味道与接缝：采用实例项目中重复实现和自调用现象；演化策略的取舍：比较同一系统的单体版本与微服务版本的差异；

接缝与特征测试的指定阅读放在做重构类实验之前；AI 辅助重构要当场对照测试套件验证正确性。

5.本章参考资料

[1] 《软件设计与体系结构》；[2] Martin《敏捷软件开发》；[3] Gamma 等《设计模式》第 1 章；[4] Fowler《重构》第 2 版第 3 章；[5] Kerievsky《重构与模式》第 3 章；[6] Feathers《修改代码的艺术》第 4、13 章；[22] Parnas 1972；[23] Lehman 1980；[24] Chikofsky 等 1990；[26] Bennett 等 2000；[28] Lenarduzzi 等 2020。

第五章：编码、测试与部署（6 学时，支撑课程目标 3、4）

1.教学重点和难点

重点：整洁代码的判据与做法；命名规范；接口契约；测试的基本要求；面向不确定性的设

计原则；代码编译与运行环境的部署。

难点：理解规范本身也是设计决策；理解 AI 验收为何不能只看测试是否通过；利用设计原则和规范来验收 AI 产出，而不是只看测试与否。

2.教学基本要求

理解：整洁代码的核心思想与主要原则；类库、程序集、命名空间三者的关系；代码评审关注的内容；提交说明的写法；测试在开发过程中的位置。

掌握：变量、函数、接口与类的命名规范及面向对象约束、异常处理、分层；软件开发文档；软件复用概念、分类和代码复用原则；接口契约中资源、错误与版本的约定；AI 产品测试的基本要求；能把代码编译并部署到运行环境。

3.教学基本内容

整洁代码：核心判据是降低后续每一次阅读和修改的成本，即改一个需求需要动几处、读多少行、是否担心碰坏别处；包括命名、函数、注释、格式、错误处理、边界封装、测试与类的设计等要求；错误抽象的成本往往高于重复代码，过度拆分同样会降低可读性，一切形式要求都服从这一判据。

编码规范：变量、函数、接口与类的命名规范；面向对象约束、异常与分层条目；大小写约定；不同的规范对同一问题的规定不同，例如一套偏重防缺陷，另一套偏重格式统一。

接口契约规范：资源命名、错误结构、版本处理；在保持向后兼容的前提下改进接口。

协作记录规范：代码评审的关注点；提交说明的写法与要求，使变更的原因能被他人独立读懂。

代码评审应关注：设计、复杂度、测试；

测试的基本要求：软件测试的级别，黑盒与白盒测试，静态与动态测试，基于控制流的覆盖率指标，软件测试 3A 原则，即 Arrange-Act-Assert；需求验收准则；AI 产品测试：评测集与对抗

用例、风险台账、放行与驳回的判定。

面向不确定性的设计：把模型预测当作不可靠组件看待，它会在未知的时刻以未知的原因出错，因此设计的目标不是消除错误，而是让系统的整体目标在出错时仍然成立。借鉴分层解耦、信息隐藏对“变化与失败”的应对思想，给出以下设计原则：（1）假设错误不可避免；（2）按后果分级选择人机分工：自动化、建议式交互、人工审批，按出错代价与用户能力选取，高风险场景保留人工回路与撤销能力；（3）护栏：用非 AI 的硬性手段限制预测的输入与输出，如取值区间限制、禁用词过滤、已知输入覆盖；（4）执行者与检查者：由独立组件或独立信号检测错误并纠正，不依赖模型自身的置信度；（5）分布外检测：输入偏离预期时拒绝预测，而不是给出低置信度结果；（6）降级：检测到组件失效时降低功能与性能以维持安全运行，而不是整体停机；（7）检测与恢复：把发现错误、纠正错误、恢复状态作为设计的一部分。

代码编译与运行环境部署：构建产物、依赖管理、配置与环境变量；把代码从开发环境部署到运行环境；部署后的验证与回滚。

AI for SE：AI 结对编程的“生成—审查—测试验证”的完整流程；AI 辅助生成测试与部署配置；验证 AI 产出。

4.教学建议

用既有系统里“实现看似正确、测试也可能通过，但违反业务约束”的例子，说明验收不能只看测试过没过；

线上预习规范条目，课堂只讲规范之间的取舍；部署环节结合 E1 的构建与运行练习，另设选开实验“测试与部署”，训练编译与部署能力。

5.本章参考资料

[12] Cwalina 等《.NET 设计规范》第 3、4 章；[13] Tulach《软件框架设计的艺术》；[30] Suh 等 2025；[31] 阿里巴巴《Java 开发手册（黄山版）》；[32] Google Java Style Guide；[33] Google

《Code Review》；[34] Microsoft REST API Guidelines；[35] Conventional Commits 1.0.0；[14] Kästner《Machine Learning in Production》；[40] CMU《Machine Learning in Production》课程与教材；[41] Stanford CS 329S 公开讲义。

六、教学安排

表 2 教学总体安排表

序号	教学内容	总学时	其　中				课外辅导/课外实践	备 注
			讲课	实验	上机	其他		
1	标准与规范体系	2	2					
2	运行环境	4	2	2				
3	体系结构设计	8	4	4				
4	软件设计、重构与系统演化	8	4	4				
5	编码、测试与部署	6	2	4				
合计		28	14	14				

表 3 实验安排表

序号	教学轮次	实验名称	学时分配	必开或选开	实验类型	分组人数
1	1	开发环境 / IntelliJ IDEA + Claude Code，接入 DeepSeek	2	选开	设计	1
2	1	棕地仓库导读 绘制 7 个 Maven 模块的结构图与依赖图；绘制调用链；提交一页导读说明（模块职责、依赖关系、构建方式、疑点清单）	2	必开	设计	1
3	1	体系结构建模与方案评价 用"4+1"视图对现有系统建模；识别分层问题（业务判断写在控制器里、事务边界、硬编码）；提出两种解耦方案并对比评价	2	必开	设计	1
4	1	规格先行与 AI 结对编程 用提示词模式把需求写成规格 + 先写验收准则；由 AI 生成失败测试与实现，人工逐处审查 diff，用执行记录与覆盖率核实测试是否守住准则，作采纳、部分采纳或驳回；提交 AI 协作记录（披露 + 对话摘要 + diff 审查记录）	2	必开	设计	1
5	1	设计模式与重构 在仓库中定位一处条件分支密集的业务代码，用提示词模式让 AI 给出重构方案（如策略模式），按模式的四个要素评价其效果与局限，并与更简单的方案比较；对照特征测试与执行记录核实 AI 的结论	2	必开	设计	1
6	2	变更影响分析 针对下发的需求（如"商品库存预警"），定位修改点、分析影响面、评估风险，并逐条核实 AI 给出的影响面判断；提交变更计划（改动文件清单、影响模块、回归测试范围）	2	必开	综合	1
7	2	规格驱动的需求变更 把模糊需求写成需求规格（用户故事 + 验收准则 + 非功能需求）；评价 AI 给出的实现在职责划分、接口兼容与编码规范上的取舍，核实其测试与执行记录；提交 ADR（架构决策记录）	2	必开	综合	1
8	2	AI 功能接入与验收 评价 AI 组件（如"自然语言查询商品"）的接入方案：护栏、置信度与来源展示、降级路径的设计取舍；实现由 AI 生成或取教师示例，依据盲评测集与对抗用例集完成验收；提交验收报告（含风险台账与放行/降级/驳回结论）	2	必开	综合	1
9	2	编码规范与接口契约审查	2	选开	综合	1

		对照两套编码规范与接口规范评价既有系统，在保持兼容的前提下评价 AI 给出的一处改进并写架构决策记录				
10	2	测试与部署 把代码编译部署到运行环境，为变更补充验收测试并达到覆盖准则	2	选开	综合	1
11	2	独立任务／在未做过的模块上独立完成诊断、评价 AI 方案与验证，按讲清楚、有依据、验证过讲清缺陷、依据、改动与证据	2	选开	综合	1

七、授课方法

采取案例教学方法，教学内容包括知识准备和实践操作两个部分，前者讲授解决问题所需思维图式和知识点，后者聚焦开发工具、运行环境和设计标准、编码规范的训练。运用自建网络教学平台开展混合式教学，线下完成案例讲解与答疑、讨论，线上实施探究式学习、自主学习。

案例教学：案例是典型工程问题，属于良构问题，即提供参考答案，供学生模仿；完成案例的过程就是解决工程问题的过程，实现知识、技能的习得与内化；案例是有机整体，彼此间既有联系也有区别，起到串联新旧知识、技能的作用；结合案例难度与当期学生水平，教学组织形式可灵活多样，如单人、小组或结对编程等；期末的综合实训采用自选案例，属于劣构问题，即没有固定答案，达到培养创造性思维目的。

采用棕地项目是本课程案例教学的亮点之一，棕地开发是以既有系统为起点的开发方式。真实的软件工作大多不是从零开始，而是在一个有历史、有技术债、约束已经定下来的系统上做修改。本课程据此设计：选一个规模较大、带着技术债的开源系统作为载体，全部实验都在它上面进行。学生先读懂它，再诊断它，然后在约束下评价改进方案，最后用规范和测试核实方案与 AI 产出是否真实可信。这样做的好处是，学生遇到的困难与工程现场一致——找不到入口、看不懂别人的设计、改一处牵动几处、遵循约束。

授课过程分前后两部分。前半部分聚焦软件设计核心活动：认识系统和它的约束，用设计原则找出问题，学会在测试保护下修改。后半部分关注变更与验证：把需求写成规格，与 AI 分工完

成变更，再拿原则和规范去验收 AI 的产出。两部分都在同一个系统上，问题和概念反复出现，认识逐步加深。

AI for SE 贯穿各个环节：与 AI 结对协作，但每一步都须留痕，并按 AI 输出的核对内容核对 AI 的输出。

课堂教学：讲解基本概念、基本原理，分析教学案例的解决思路与技术手段，通过对作业的点评、讨论，师生互动，训练学生的批判性思维，激发学生的创新意识，培养学生分析问题和解决问题的能力。

思维训练：按照面向问题解决的逻辑来组织教学内容，培养学生解决复杂问题的能力。每个教学案例学生均经过“模仿-反思-创新”三个阶段，指导学生开展批判、分析和评价活动，反思思维过程，尤其是不同技术比较、设计方案的取舍、AI 输出的核对等，使得运用分析、评价、综合和创造等高阶思维能力成为自觉。

信息化教学：采用“线上预习、课堂实操、实践讲解、现场演示”的完整流程，完成预习做好实验前知识准备，课堂实操老师及时答疑、减少课后负担，实践讲解回顾知识点，现场演示比对方案取舍。网络课堂发布预习任务与实践作业，提供参考材料，组织课堂演示与投票；定期导出预习正确率、签到情况与提交率，用于学情分析并调整教学。通过完成实践讲解、现场演示达到学习效能自我感知；任务与成绩挂钩，实现过程性评价；借助学习通 APP 搭建师生互动桥梁，即时答疑解惑，解决上课没听清、下课找不到老师问的问题。

八、考核办法

1．考核方式

总评成绩 = 网络课堂成绩 40% + 项目实践（作业）成绩 60%，前者采用过程性评价，百分制，后者为终结性评价，百分制。

2.评分标准

网络课堂成绩 = 线上预习 10% + 签到 10% + 课程网站访问 10% + 课堂演示 10%；其中签到以投票答题计，评分标准见表 4。

表 4 网络课堂评分标准

等级	线上预习与测验	签到（投票答题）	课程网站访问	课堂演示
优秀	按时完成全部预习，回答正确	签到满 10 次	访问满 300 次	每次按要求参与
良好	按时完成 3/4 预习，回答正确	签到满 8 次	访问满 240 次	未参与不超过 2 次
中等	按时完成 2/4 预习，回答正确	签到满 6 次	访问满 180 次	未参与不超过 4 次
及格	按时完成 1/4 预习，回答正确	签到满 4 次	访问满 120 次	未参与不超过 6 次
不及格	未按时完成 1/4 预习，或回答不正确	签到未满 4 次	访问不满 120 次	未按要求参与

线上任务均设有截止日期，过期后无法提交。

（1）线上预习，内容为核对 AI 给出的结论与代码（核对引用是否存在、核对运行时事实、区分原理与实现、拿原则与规范作依据），每次实验前完成，采用客观题，系统自动判分，未做计零分，总分取平均分；

（2）签到，理论课与实验课均以投票答题计到场：每次课就本次内容出一道小题，答对计该次签到；签到 1 次计 1 分，10 次以上为满分；

（3）课程网站访问，300 次及以上得满分，次数不足按比例扣分；

（4）课堂演示，实验课开头不超过 15 分钟，从最近一次作业终评的前 30% 与后 30% 中选取学生实名演示；参与演示与投票即计该次，被选中上台演示者按讲清楚、有依据、验证过三项评分标准评分。

作品讲解是项目实践的主要形式：每次作业提交不超过 2 分钟的录屏与一页说明，按讲清楚、有依据、验证过三项评分标准评分，见表 5；自评、教师评分与同学投票使用同一套标准，被选中现场演示的学生，该次演示成绩由教师评分与同学投票各占一半。

作品讲解、自评、教师评分与同学投票使用同一套标准。评分标准：讲清楚、有依据、验证过，具体地：①讲清楚：观察现象、分析问题、拆分问题，说清向 AI 提了什么问题，并给出 文件:行号，能说出调用者或数据流；②有依据：说明评价设计方案所依据的原则、标准与规范，不同方案取舍的理由，以及对 AI 产出采纳、部分采纳或驳回的依据；③验证过：用断点、代码、执行记录、测试记录核实 AI 的答案是否真实可信，说明证明了什么、没有证明什么。

表 5 作品讲解评分标准

等级	讲清楚	有依据	验证过
优秀	用自己的话讲清观察到的现象、分析出的问题与拆分方式，能说出调用者或数据流	给出评价所依据的原则、标准与取舍理由，并给出文件:行号	用断点、代码、执行记录或测试记录核实 AI 的答案，说明证明了什么
合格	需要看稿或经提示才讲清	有原则，没有证据	有记录但没说明证明了什么
不及格	只能复述 AI 的话或讲不出	没有依据	没有核实，或只说"AI 说对了"

3．AI 使用与学术诚信规范

允许并鼓励在项目实践中使用生成式 AI 工具，但须遵守：

（1）在提交物中标注使用了 AI 的部分及所用工具；

（2）能够向教师解释 AI 产出的每一处内容，答辩时随机抽查讲解；

（3）对提交物的正确性与质量负全部责任；

（4）作品讲解的录屏须由本人完成，并演示核实 AI 答案的过程（断点、执行记录或测试运

行）。

（5）AI 检测器的结论不作为认定违规的依据，核验依靠作品讲解、现场演示和期末独立展示。

（6）未标注使用 AI、无法解释产出内容或以 AI 产出冒充本人完成的，按学术不诚信处理。

九、教材及参考书

1.教材与专著：

[1] 董威，文艳军，陈振邦. 软件设计与体系结构[M]. 2 版. 北京：高等教育出版社，2017. ISBN 9787040486308.（选用教材）

[2] Martin R C. 敏捷软件开发：原则、模式与实践[M]. 邓辉，译. 北京：清华大学出版社，2003.

[3] Gamma E, Helm R, Johnson R, Vlissides J. 设计模式：可复用面向对象软件的基础[M]. 李英军，马晓星，蔡敏，刘建中，译. 北京：机械工业出版社，2000.

[4] Fowler M. 重构：改善既有代码的设计（第 2 版）[M]. 熊节，林从羽，译. 北京：人民邮电出版社，2019.

[5] Kerievsky J. 重构与模式（修订版）[M]. 杨光，刘基诚，译. 北京：人民邮电出版社，2013.

[6] Feathers M C. 修改代码的艺术[M]. 刘未鹏，译. 北京：人民邮电出版社，2007.

[7] Fowler M. 企业应用架构模式[M]. 王怀民，周斌，译. 北京：机械工业出版社，2010.

[8] Ford N, Parsons R, Kua P. 演进式架构[M]. 周训杰，译. 北京：人民邮电出版社，2019.

[9] Evans E. 领域驱动设计：软件核心复杂性应对之道（修订版）[M]. 赵俐，盛海艳，刘霞，译. 北京：人民邮电出版社，2016.

[10] Vernon V. 实现领域驱动设计[M]. 滕云，译. 北京：电子工业出版社，2014.

[11] Newman S. 微服务设计[M]. 崔力强，张骏，译. 北京：人民邮电出版社，2016.

[12] Cwalina K, Abrams B. .NET 设计规范：约定、惯用法与模式（第 2 版）[M]. 葛子昂，译.

北京：人民邮电出版社，2010.

[13] Tulach J. 软件框架设计的艺术[M]. 王磊，朱兴，译. 北京：人民邮电出版社，2011.

[14] Kästner C. Machine Learning in Production: From Models to Products[M]. Cambridge, MA: MIT Press, 2025. 在线全文：https://mlip-cmu.github.io/book/.（Architecture 与 Planning for Mistakes 章节对应第三章）

[15] Washizaki H. Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0[M]. IEEE Computer Society, 2024. https://www.swebok.org.

2.国家标准与国际标准：

[16] GB/T 25000.10-2016 系统与软件工程 系统与软件质量要求和评价（SQuaRE） 第 10 部分：系统与软件质量模型[S].

[17] GB/T 8566-2022 系统与软件工程 软件生存周期过程[S].

[18] GB/T 8567-2006 计算机软件文档编制规范[S].

[19] ISO/IEC 25010:2023 Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model[S].

[20] ISO/IEC/IEEE 12207:2026 Systems and software engineering—Software life cycle processes[S].（代替 2026 年 4 月 29 日撤回的 12207:2017）

[21] Mell P, Grance T. The NIST Definition of Cloud Computing: NIST SP 800-145[S]. Gaithersburg: NIST, 2011. DOI: 10.6028/NIST.SP.800-145.

3.论文：

[22] Parnas D L. On the criteria to be used in decomposing systems into modules[J]. Communications of the ACM, 1972, 15(12): 1053-1058. DOI: 10.1145/361598.361623.

[23] Lehman M M. Programs, life cycles, and laws of software evolution[J]. Proceedings of the IEEE,

1980, 68(9): 1060-1076. DOI: 10.1109/PROC.1980.11805.

[24] Chikofsky E J, Cross J H. Reverse engineering and design recovery: a taxonomy[J]. IEEE Software, 1990, 7(1): 13-17. DOI: 10.1109/52.43044.

[25] Kruchten P. Architectural blueprints—the “4+1” view model of software architecture[J]. IEEE Software, 1995, 12(6): 42-50. DOI: 10.1109/52.469759.

[26] Bennett K H, Rajlich V T. Software maintenance and evolution: a roadmap[C]//Proceedings of the Conference on the Future of Software Engineering. 2000: 73-87. DOI: 10.1145/336512.336534.

[27] Le D M, Behnamghader P, Garcia J, et al. An empirical study of architectural change in open-source software systems[C]//Proceedings of the 12th Working Conference on Mining Software Repositories (MSR 2015). 2015: 235-245. DOI: 10.1109/MSR.2015.29.

[28] Lenarduzzi V, Lomio F, Saarimäki N, Taibi D. Does migrating a monolithic system to microservices decrease the technical debt?[J]. Journal of Systems and Software, 2020, 169: 110710. DOI: 10.1016/j.jss.2020.110710.

[29] White J, Fu Q, Hays S, et al. A prompt pattern catalog to enhance prompt engineering with ChatGPT[EB/OL]. arXiv: 2302.11382, 2023.

[30] Suh H, Tafreshipour M, Li J, Bhattiprolu A, Ahmed I. An empirical study on automatically detecting AI-generated source code: how far are we?[C]//Proceedings of the 47th International Conference on Software Engineering (ICSE 2025). 2025. arXiv: 2411.04299.

4.线上资源：

[31] 阿里巴巴集团. Java 开发手册（黄山版）[EB/OL]. 2022. https://github.com/alibaba/p3c.

[32] Google. Google Java Style Guide[EB/OL]. https://google.github.io/styleguide/javaguide.html.

[33] Google. Google Engineering Practices: Code Review[EB/OL]. https://google.github.io/eng-

practices/review/.

[34] Microsoft. Microsoft REST API Guidelines[EB/OL]. https://github.com/microsoft/api-guidelines.

[35] Conventional Commits. Conventional Commits 1.0.0[EB/OL]. https://www.conventionalcommits.org/en/v1.0.0/.

[36] Fowler M. GUI Architectures[EB/OL]. 2006. https://martinfowler.com/eaaDev/uiArchs.html.

[37] Fowler M. Inversion of Control Containers and the Dependency Injection pattern[EB/OL]. 2004. https://martinfowler.com/articles/injection.html.

[38] Microsoft. Model-View-ViewModel (Enterprise Application Patterns Using .NET MAUI)[EB/OL]. https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm.

[39] Spring. Spring Framework Reference: Dependency Injection[EB/OL]. https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html.

[40] Carnegie Mellon University. Machine Learning in Production: 课程讲义、幻灯片与教材全文[EB/OL]. https://mlip-cmu.github.io/；教材章节 https://mlip-cmu.github.io/book/. CC 协议开源，对应第三章。

[41] Stanford University. CS 329S: Machine Learning Systems Design [EB/OL]. https://stanford-cs329s.github.io/syllabus.html.

[42] Vanderbilt University. Prompt Engineering for ChatGPT[EB/OL]. Coursera. https://www.coursera.org/learn/prompt-engineering.

[43] UC Berkeley. Large Language Model Agents[EB/OL]. https://llmagents-learning.org/.（系列课程现迁至 https://agenticai-learning.org/）

[44] Northeastern University CS4530. Policy on Use of AI (Spring 2026)[EB/OL]. https://neu-se.github.io/CS4530-Spring-2026/policies/ai.

[45] macrozheng. mall[CP/OL]. https://github.com/macrozheng/mall. 棕地项目。

[46] macrozheng. mall-swarm[CP/OL]. https://github.com/macrozheng/mall-swarm. 棕地项目的系统演化案例。

[47] 课程组. AI 协作与产出验收规范[Z]. 课程讲义附录.

[48] 课程组. mall / mall-swarm 产品拆解[Z]. 课程读物.

[49] Nygard M. Documenting Architecture Decisions[EB/OL]. 2011. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

系室：软件工程系　　执笔人：李萌　　系室审核人：蒋良卫